

编程语言，A 部分

- ✓

列表功能

视频 • 11分钟
- ✓

让表达式

视频 • 7分钟
- ✓

嵌套函数

视频 • 7分钟
- ✓

让与效率

视频 • 10分钟
- ✓

选项

视频 • 9分钟
- ✓

布尔运算和比较运算

视频 • 7分钟
- ✓

无突变的好处

视频 • 7分钟
- ✓

可选：Java 突变

视频 • 4分钟
- ✓

语言的碎片

视频 • 6分钟

作业 1

- ✓

作业 1（自动评分器）

编程作业 • 成绩：100%
- 作业 1 同行评估详细指南

阅读材料
- ✓

作业 1

互评作业 • 结果待处理
- 作业 1

审阅同学的作业

社区提供的资源

- ✓

作业 1 的提示和注意事项

阅读材料 • 5分钟

- ✓

额外练习题

阅读材料

单元 4

第 2 节和作业 2

单元 5

第 3 节和作业 3 -- 以及课程动机

单元 6

第 4 部分和 A 部分考试



0:01 / 8:57



1x



14. 编写函数`splitup : int list -> int list * int list`，在给定整数列表的情况下，创建两个整数列表，一个包含非负数条目，另一个包含负数条目。必须保留相对顺序：所有非负数项必须以与它们在原始列表中相同的顺序出现，负数项也是如此。
15. 编写前一个函数的`splitAt : int list * int -> int list * int list` 版本，该版本多了一个 "阈值 "参数，并使用该参数代替 0 作为两个结果列表的分界点。
16. 编写一个函数`isSorted : int list -> boolean`，在给定整数列表的情况下，确定列表是否按递增顺序排序。
17. 编写一个函数`isAnySorted : int list -> boolean`，在给定一个整数列表的情况下，确定该列表是按递增顺序排序还是按递减顺序排序。
18. 编写一个函数`sortedMerge : int list * int list -> int list`，接收从小到大排序的两个整数列表，并将它们合并为一个排序列表。例如：
`sortedMerge ([1,4,7], [5,8,9]) = [1,4,5,7,8,9]`。
19. 编写一个排序功能`qsort : int list -> int list`，其工作原理如下：取出第一个元素，将其作为`splitAt` 的 "阈值"。然后对`splitAt` 产生的两个列表进行递归排序。最后将两个列表合并。(不要忘了你取出的那个元素，它需要在某个时候重新加入)。您可以使用`sortedMerge` 来完成 "合并 "部分，但不需要这样做，因为一个列表中的所有数字都小于另一个列表中的所有数字)。
20. 编写一个函数`divide : int list -> int list * int list`，接收一个整数列表，并通过在两个列表中交替使用元素来生成两个列表。 例如：
`divide ([1,2,3,4,5,6,7]) = ([1,3,5,7], [2,4,6])`。
21. 编写另一个排序功能`not_so_quick_sort : int list -> int list`，其工作原理如下：给定初始整数列表，使用除法将其分成两个列表，然后递归对这两个列表进行排序，然后使用 `sortedMerge` 将它们合并在一起。
22. 编写一个函数`fullDivide : int * int -> int * int`，在给定两个数字`k` 和`n` 的情况下，尝试将`k` 平均分成`n` 尽可能多的次数，并返回一对`(d, n2)`，其中`d`是次数，而`n2` 是所有这些分割后得到的`n`。示例：`fullDivide (2, 40) = (3, 5)`，因为 $2*2*2*5 = 40$ 和 `fullDivide((3,10)) = (0, 10)` ，因为3 不能整除10。
23. 使用`fullDivide` 写一个函数`factorize : int -> (int * int) list`，给定一个数`n`，返回一个数对列表`(d, k)`，其中`d`是除以`n`的质数，`k`是符合的次数。这些数对应该按照质因数的递增顺序排列，当考虑的除数超过`n`的平方根时，过程应该停止。如果确保下一步都使用`fullDivide` 给出的还原数`n2`，那么应该不需要测试除数是否是质数：如果一个数除以`n`，那么它一定是质数（如果它有质因数，那么它们应该是`n`的早期质因数，因此可以更早地还原）。示例：
`factorize(20) = [(2,2), (5,1)]`;`factorize(36) = [(2,2), (3,2)]`;
`factorize(1) = []`。
24. 编写一个函数`multiply : (int * int) list -> int`，在给定数字`n`的因数分解时，如上一问题所述，计算出数字`n`。因此，这样做的结果应该与`factorize` 相反。
25. 挑战（难）：编写一个函数`all_products : (int * int) list -> int list`，在给定 `factorize` 的因式分解结果列表的同时，创建一个列表，列出使用部分或全部质因数生成的所有可能产品，但数量不得超过这些质因数的可用次数。该列表最终应包含产生该列表的`n` 数字的所有除数。示例: `all_products([(2,2), (5,1)]) = [1,2,4,5,10,20]`。为了增加挑战性，递归过程应按此顺序返回数字，而不是事后再排序。